

Q1) Write a C program to:

- Illustrates the fork-join pattern using OpenMP's parallel directive.**
- Illustrates the fork-join pattern using multiple OpenMP parallel directives and changing the number of threads two ways.**
- Illustrates the single-program-multiple-data (SPMD) pattern using two basic OpenMP commands.**

Code:

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main() {
5
6     // Illustrates the fork-join pattern using OpenMP's parallel directive.
7     printf("Reg No: 20968040, Name: Devarjya\n");
8     printf("Fork-Join\n");
9     #pragma omp parallel
10    {
11        printf("Thread %d inside parallel region\n", omp_get_thread_num());
12    }
13    printf("\n");
14
15    // Illustrates the fork-join pattern using multiple OpenMP parallel directives and changing the
16    printf("Changing Thread Counts\n");
17
18    #pragma omp parallel num_threads(3)
19    {
20        #pragma omp single
21        printf("1. Using num_threads(3) clause: Active threads = %d\n", omp_get_num_threads());
22    }
23
24    omp_set_num_threads(2);
25    #pragma omp parallel
26    {
27        #pragma omp single
28        printf("2. Using omp_set_num_threads(2): Active threads = %d\n", omp_get_num_threads());
29    }
30    printf("\n");
31
32    // Illustrates the single-program-multiple-data (SPMD) pattern using two basic OpenMP commands
33    printf("SPMD Pattern\n");
34    #pragma omp parallel num_threads(4)
35    {
36        int tid = omp_get_thread_num();
37        printf("SPMD: Hello from thread %d doing its own task.\n", tid);
38    }
39
40    return 0;
41 }
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 2$ gcc q1.c -o q1 -fopenmp
pcap1@PC-36:~/Documents/240968040/week 2$ ./q1
Reg No: 20968040, Name: Devarjya
Fork-Join
Thread 7 inside parallel region
Thread 1 inside parallel region
Thread 12 inside parallel region
Thread 25 inside parallel region
Thread 4 inside parallel region
Thread 27 inside parallel region
Thread 18 inside parallel region
Thread 0 inside parallel region
```

Q2) Write a OpenMP program to calculate (i , x) for all the threads where i is an integer value and tid is the thread_id.

Code:

```
1#include <stdio.h>
2#include <omp.h>
3
4int calc(int i, int x)
5{
6    return i*x;
7}
8int main()
9{
10    printf("Reg No: 240968040, Name: Devarjya\n");
11    int x = 2;
12    #pragma omp parallel
13    {
14        int tid = omp_get_thread_num();
15        int result = calc(x, tid);
16
17        printf("i = %d, x = %d, result = %d\n", x, tid, result);
18    }
19    return 0;
20}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 2$ ./q2
Reg No: 240968040, Name: Devarjya
i = 2, x = 5, result = 10
i = 2, x = 10, result = 20
i = 2, x = 0, result = 0
i = 2, x = 26, result = 52
i = 2, x = 22, result = 44
i = 2, x = 21, result = 42
i = 2, x = 1, result = 2
i = 2, x = 4, result = 8
i = 2, x = 27, result = 54
i = 2, x = 3, result = 6
i = 2, x = 25, result = 50
i = 2, x = 6, result = 12
i = 2, x = 7, result = 14
i = 2, x = 17, result = 34
i = 2, x = 2, result = 4
i = 2, x = 23, result = 46
i = 2, x = 24, result = 48
i = 2, x = 14, result = 28
i = 2, x = 15, result = 30
i = 2, x = 16, result = 32
i = 2, x = 12, result = 24
i = 2, x = 13, result = 26
i = 2, x = 11, result = 22
i = 2, x = 20, result = 40
i = 2, x = 18, result = 36
i = 2, x = 9, result = 18
i = 2, x = 19, result = 38
i = 2, x = 8, result = 16
```

Q3) Write a OpenMP program that performs the sum of even numbers and odd numbers in a given input array. Create a separate thread to perform the sum of even numbers and odd numbers.

Code:

```
1#include <stdio.h>
2#include <omp.h>
3
4int main()
5{
6    int sum_even=0;
7    int sum_odd=0; int n;
8
9    printf("Reg No: 240968040, Name: Devarjya\n");
10   printf("Enter size of array\n");
11   scanf("%d", &n);
12   int a[n];
13   printf("Enter elements:\n");
14
15   for (int i=0; i<n; i++)
16   {
17       scanf("%d", &a[i]);
18   }
19   omp_set_num_threads(2);
20
21   #pragma omp parallel shared(a, n, sum_even, sum_odd)
22   {
23       int tid = omp_get_thread_num();
24       if(tid==0)
25       {
26           for(int i=0; i<n; i++)
27               if(a[i]%2==0) sum_even += a[i];
28
29           printf("Thread %d calculated even sum %d\n", tid, sum_even);
30       }
31
32       if(tid==1)
33       {
34           for(int i=0; i<n; i++)
35               if(a[i]%2!=0) sum_odd += a[i];
36
37           printf("Thread %d calculated odd sum %d\n", tid, sum_odd);
38       }
39   }
40   return 0;
41 }
```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 2$ gcc q3.c -o q3 -fopenmp
pcap1@PC-36:~/Documents/240968040/week 2$ ./q3
Reg No: 240968040, Name: Devarjya
Enter size of array
4
Enter elements:
1
2
3
4
Thread 0 calculated even sum 6
Thread 1 calculated odd sum 4

```

Q4) Write a OpenMP program to implement all the four basic operations of a calculator (Add, Sub, Mul, Div). Create a separate thread to perform the operations

Code:

```

1#include <stdio.h>
2#include <omp.h>
3
4int main() {
5    double a, b;
6    printf("Reg No: 240968040, Name: Devarjya\n");
7
8    printf("Enter two numbers: ");
9    scanf("%lf %lf", &a, &b);
10
11    #pragma omp parallel num_threads(4)
12    {
13        int tid = omp_get_thread_num();
14
15        if (tid == 0) {
16            printf("Addition: %.2lf (Executed by thread %d)\n", a + b, tid);
17        } else if (tid == 1) {
18            printf("Subtraction: %.2lf (Executed by thread %d)\n", a - b, tid);
19        } else if (tid == 2) {
20            printf("Multiplication: %.2lf (Executed by thread %d)\n", a * b, tid);
21        } else if (tid == 3) {
22            printf("Division: %.2lf (Executed by thread %d)\n", a / b, tid);
23        }
24    }
25
26    return 0;
27}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 2$ ./q4
Reg No: 240968040, Name: Devarjya
Enter two numbers: 5
4
Addition: 9.00 (Executed by thread 0)
Division: 1.25 (Executed by thread 3)
Multiplication: 20.00 (Executed by thread 2)
Subtraction: 1.00 (Executed by thread 1)

```

Q5) Write a OpenMP program for generating prime numbers from a given starting number to the given ending number.

Code:

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 int is_prime(int n) {
5     if (n <= 1) return 0;
6     for (int i = 2; i * i <= n; i++) {
7         if (n % i == 0) return 0;
8     }
9     return 1;
10 }
11
12 int main() {
13     int start, end;
14
15     printf("Reg No: 240968040, Name: Devarjya\n");
16
17     printf("Enter start and end: ");
18     scanf("%d %d", &start, &end);
19
20     printf("Primes:\n");
21
22     #pragma omp parallel for
23
24     for (int i = start; i <= end; i++) {
25         int tid = omp_get_thread_num();
26         if (is_prime(i)) {
27             printf("%d found %d\n", tid, i);
28         }
29     }
30
31     return 0;
32 }
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 2$ ./q5
Reg No: 240968040, Name: Devarjya
Enter start and end: 1 20
Primes:
6 found 7
2 found 3
1 found 2
4 found 5
18 found 19
16 found 17
12 found 13
10 found 11
```

Q6) Write a program in OpenMP to toggle the character of a given character array indexed by the thread_id. Print the corresponding Thread_id.

Example: suppose the string is “HeLLo”, then the output should be “hEllo”

Code:

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <omp.h>
4
5 int main() {
6     char str[] = "Hello Everyone";
7     printf("Reg No: 240968040, Name: Devarjya\n");
8     printf("Original string: %s\n", str);
9
10    int len = strlen(str);
11
12    omp_set_num_threads(len);
13
14    #pragma omp parallel
15    {
16        int tid = omp_get_thread_num();
17
18        if (tid < len) {
19            char c = str[tid];
20            if (c >= 'a' && c <= 'z') {
21                str[tid] = c - 32;
22            } else if (c >= 'A' && c <= 'Z') {
23                str[tid] = c + 32;
24            }
25
26            printf("Thread %d toggled character at index %d\n", tid, tid);
27        }
28    }
29
30    printf("Output: %s\n", str);
31    return 0;
32 }
33 }

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 2$ ./q6
Reg No: 240968040, Name: Devarjya
Original string: Hello Everyone
Thread 0 toggled character at index 0
Thread 3 toggled character at index 3
Thread 8 toggled character at index 8
Thread 2 toggled character at index 2
Thread 5 toggled character at index 5
Thread 1 toggled character at index 1
Thread 7 toggled character at index 7
Thread 13 toggled character at index 13
Thread 6 toggled character at index 6
Thread 10 toggled character at index 10
Thread 11 toggled character at index 11
Thread 12 toggled character at index 12
Thread 9 toggled character at index 9
Thread 4 toggled character at index 4
Output: hELLO eVERYONE

```

Q7) Write a program using OpenMP to compute the Fibonacci number for the following arrays of numbers: A={10, 13, 5, 6}. Create a separate thread to perform the operations.

Code:

```
#include <stdio.h>
#include <omp.h>

long long fibonacci(int n) {
    if (n <= 0) return 0;
    if (n == 1) return 1;
    long long a = 0, b = 1, c = 0;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return c;
}

int main() {
    int A[] = {10, 13, 5, 6};
    int n = 4;
    long long F[4];
    double start, time_taken;

    printf("Reg No: 240968040, Name: Devarjya\n");
    printf("Array A: {10, 13, 5, 6}\n\n");

    start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        F[i] = fibonacci(A[i]);

        printf("Thread %d computed F[%d] = %lld for A[%d] = %d\n", tid, i, F[i], i, A[i]);
    }

    time_taken = omp_get_wtime() - start;
    printf("\nTime taken: %lf seconds\n", time_taken);

    printf("\nFinal Fibonacci Array F: {");
    for (int i = 0; i < n; i++) {
        printf("%lld%s", F[i], (i == n - 1) ? "" : ", ");
    }
}
```



```
    printf("}\n");

    return 0;
}
```

Output

```
pcap1@PC-36:~/Documents/240968040/week 2$ export OMP_NUM_THREADS=1
pcap1@PC-36:~/Documents/240968040/week 2$ gcc q7.c -o q7 -fopenmp
pcap1@PC-36:~/Documents/240968040/week 2$ ./q7
Reg No: 240968040, Name: Devarjya
Array A: {10, 13, 5, 6}

Thread 0 computed F[0] = 55 for A[0] = 10
Thread 0 computed F[1] = 233 for A[1] = 13
Thread 0 computed F[2] = 5 for A[2] = 5
Thread 0 computed F[3] = 8 for A[3] = 6

Time taken: 0.000035 seconds

Final Fibonacci Array F: {55, 233, 5, 8}
pcap1@PC-36:~/Documents/240968040/week 2$ export OMP_NUM_THREADS=2
pcap1@PC-36:~/Documents/240968040/week 2$ ./q7
Reg No: 240968040, Name: Devarjya
Array A: {10, 13, 5, 6}

Thread 0 computed F[0] = 55 for A[0] = 10
Thread 0 computed F[1] = 233 for A[1] = 13
Thread 1 computed F[2] = 5 for A[2] = 5
Thread 1 computed F[3] = 8 for A[3] = 6

Time taken: 0.000264 seconds

Final Fibonacci Array F: {55, 233, 5, 8}
pcap1@PC-36:~/Documents/240968040/week 2$ export OMP_NUM_THREADS=4
pcap1@PC-36:~/Documents/240968040/week 2$ ./q7
Reg No: 240968040, Name: Devarjya
Array A: {10, 13, 5, 6}

Thread 0 computed F[0] = 55 for A[0] = 10
Thread 1 computed F[1] = 233 for A[1] = 13
Thread 2 computed F[2] = 5 for A[2] = 5
Thread 3 computed F[3] = 8 for A[3] = 6

Time taken: 0.000618 seconds

Final Fibonacci Array F: {55, 233, 5, 8}
```

```
pcap1@PC-36:~/Documents/240968040/week 2$ export OMP_NUM_THREADS=6
pcap1@PC-36:~/Documents/240968040/week 2$ ./q7
Reg No: 240968040, Name: Devarjya
Array A: {10, 13, 5, 6}

Thread 0 computed F[0] = 55 for A[0] = 10
Thread 1 computed F[1] = 233 for A[1] = 13
Thread 2 computed F[2] = 5 for A[2] = 5
Thread 3 computed F[3] = 8 for A[3] = 6

Time taken: 0.000699 seconds

Final Fibonacci Array F: {55, 233, 5, 8}
pcap1@PC-36:~/Documents/240968040/week 2$ export OMP_NUM_THREADS=8
pcap1@PC-36:~/Documents/240968040/week 2$ ./q7
Reg No: 240968040, Name: Devarjya
Array A: {10, 13, 5, 6}

Thread 0 computed F[0] = 55 for A[0] = 10
Thread 1 computed F[1] = 233 for A[1] = 13
Thread 3 computed F[3] = 8 for A[3] = 6
Thread 2 computed F[2] = 5 for A[2] = 5

Time taken: 0.000768 seconds

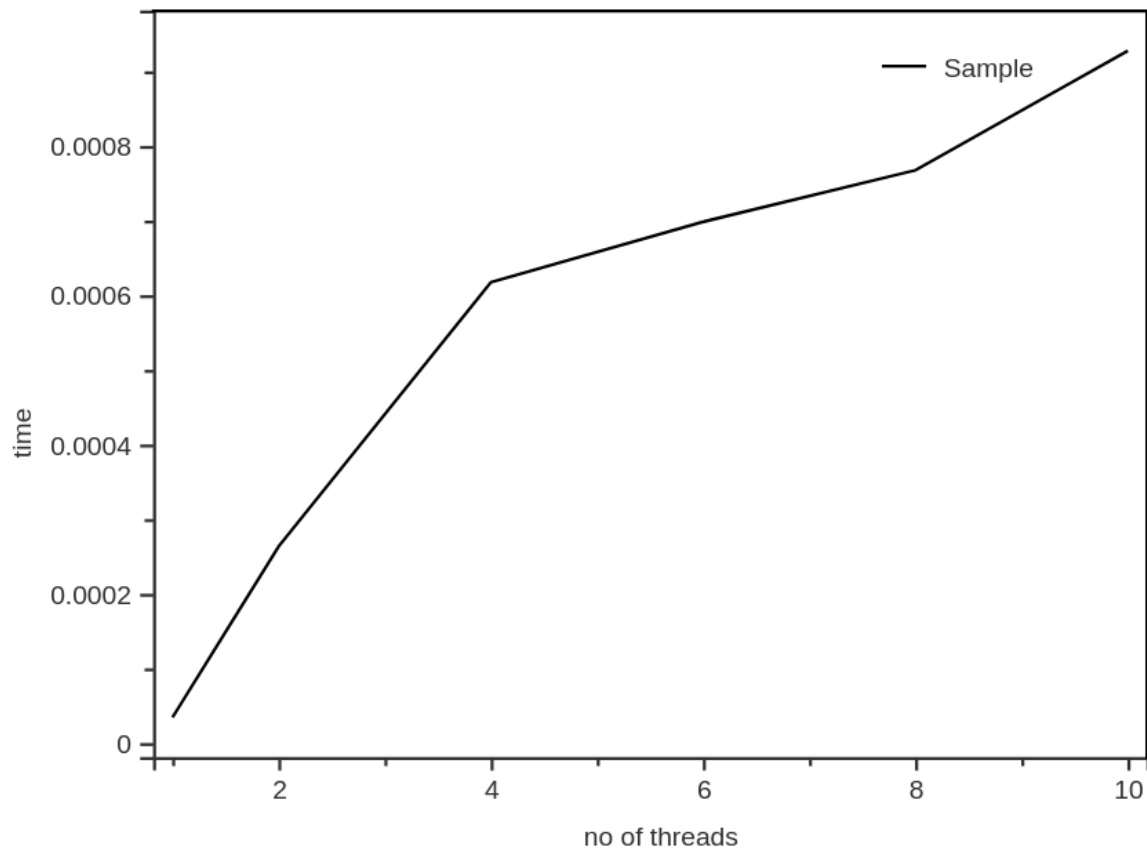
Final Fibonacci Array F: {55, 233, 5, 8}
pcap1@PC-36:~/Documents/240968040/week 2$ export OMP_NUM_THREADS=10
pcap1@PC-36:~/Documents/240968040/week 2$ ./q7
Reg No: 240968040, Name: Devarjya
Array A: {10, 13, 5, 6}

Thread 0 computed F[0] = 55 for A[0] = 10
Thread 3 computed F[3] = 8 for A[3] = 6
Thread 1 computed F[1] = 233 for A[1] = 13
Thread 2 computed F[2] = 5 for A[2] = 5

Time taken: 0.000928 seconds

Final Fibonacci Array F: {55, 233, 5, 8}
```

Plot:



Q8) Write an OpenMP program to perform Matrix times vector multiplication. Vary the matrix and vector size and analyze the speedup and efficiency of the parallelized code.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int rows, cols;

    printf("Reg No: 240968040, Name: Devarjya\n");

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    int **mat = (int **)malloc(rows * sizeof(int *));
    for (int i = 0; i < rows; i++)
        mat[i] = (int *)malloc(cols * sizeof(int));

    int *vec = (int *)malloc(cols * sizeof(int));
    int *result = (int *)calloc(rows, sizeof(int));

    printf("Enter matrix elements:\n");
```

```

for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        scanf("%d", &mat[i][j]);

printf("Enter vector elements:\n");
for (int i = 0; i < cols; i++)
    scanf("%d", &vec[i]);

double start = omp_get_wtime();

#pragma omp parallel for
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        result[i] += mat[i][j] * vec[j];
    }
}

double end = omp_get_wtime();

printf("\nResult Vector:\n");
for (int i = 0; i < rows; i++)
    printf("%d ", result[i]);

printf("\n\nExecution Time = %f seconds\n", end - start);

for (int i = 0; i < rows; i++)
    free(mat[i]);
free(mat);
free(vec);
free(result);

return 0;
}

```

Output:

```
indrath@indr4th ~> ./q8
Reg No: 240968040, Name: Devarjya
Enter number of rows and columns: 3 3
Enter matrix elements:
1 2 3
1 2 3
1 2 3
Enter vector elements:
1 2 3

Result Vector:
14 14 14
```

```
Execution Time = 0.000652 seconds
indrath@indr4th ~> ./q8
Reg No: 240968040, Name: Devarjya
Enter number of rows and columns: 4 4
Enter matrix elements:
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
Enter vector elements:
1 2 3 4

Result Vector:
30 30 30 30

Execution Time = 0.000686 seconds
```

```
indrath@indr4th ~> ./q8
Reg No: 240968040, Name: Devarjya
Enter number of rows and columns: 5 5
Enter matrix elements:
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
Enter vector elements:
1 2 3 4 5

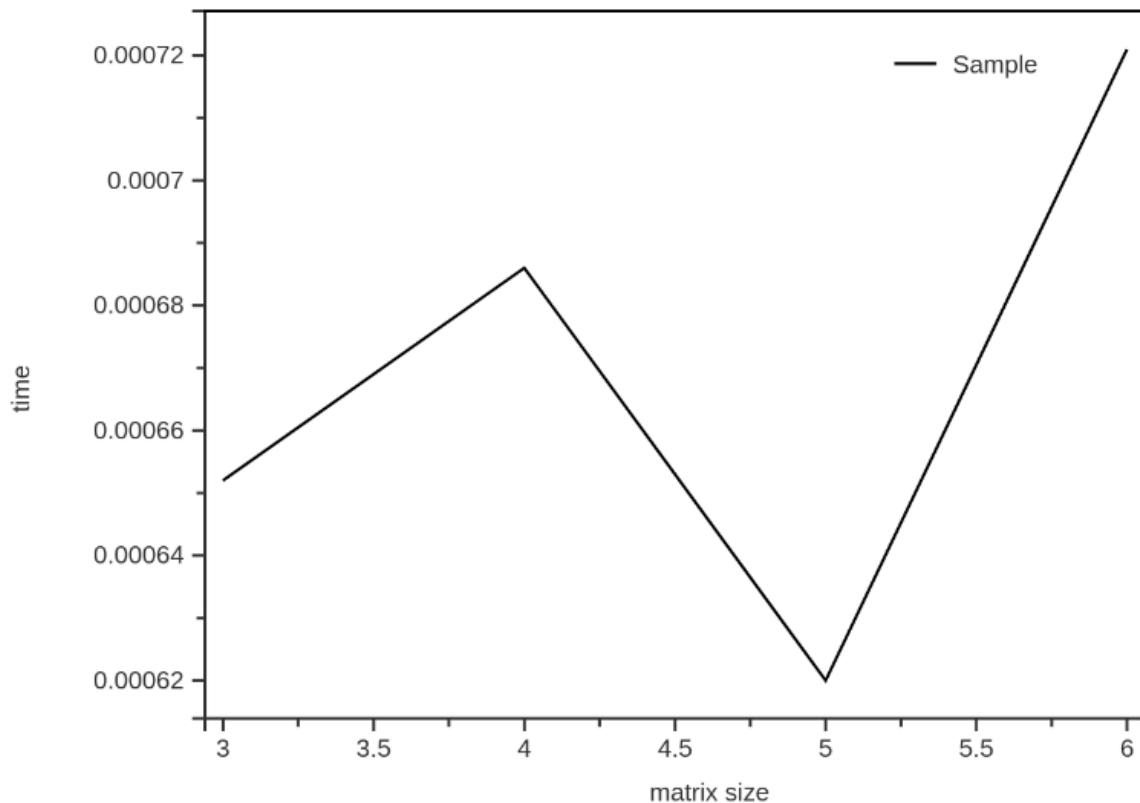
Result Vector:
55 55 55 55 55

Execution Time = 0.000620 seconds
indrath@indr4th ~> ./q8
Reg No: 240968040, Name: Devarjya
Enter number of rows and columns: 6 6
Enter matrix elements:
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
Enter vector elements:
1 2 3 4 5 6

Result Vector:
91 91 91 91 91 91

Execution Time = 0.000721 seconds
```

Plot:



Q9) Write an OpenMp program to read a matrix A of size 5x5. It produces a resultant matrix B of size 5x5. It sets all the principal diagonal elements of B matrix with 0. It replaces each row elements in the B matrix in the following manner. If the element is below the principal diagonal it replaces it with the maximum value of the row in the A matrix having the same row number of B. If the element is above the principal diagonal it replaces it with the minimum value of the row in the A matrix having the same row number of B. Analyze the speedup and efficiency of the parallelized code.

Code:

```
#include <stdio.h>
#include <omp.h>
```

```
int main() {
    int A[5][5];
    int B[5][5];

    printf("Reg No: 240968040, Name: Devarjya\n");

    printf("Enter elements of 5x5 matrix A:\n");
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            scanf("%d", &A[i][j]);
        }
    }

    double start = omp_get_wtime();

    #pragma omp parallel for
```

```

for (int i = 0; i < 5; i++) {

    int max_val = A[i][0];
    int min_val = A[i][0];

    for (int j = 1; j < 5; j++) {
        if (A[i][j] > max_val)
            max_val = A[i][j];

        if (A[i][j] < min_val)
            min_val = A[i][j];
    }

    for (int j = 0; j < 5; j++) {
        if (i == j)
            B[i][j] = 0;
        else if (i > j)
            B[i][j] = max_val;
        else
            B[i][j] = min_val;
    }
}

double end = omp_get_wtime();

printf("\nResultant Matrix B:\n");
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        printf("%d\t", B[i][j]);
    }
    printf("\n");
}

printf("\nExecution Time = %f seconds\n", end - start);

return 0;
}

```


Output:

```
indrath@indr4th ~-> export OMP_NUM_THREADS=1
indrath@indr4th ~-> ./q9
Reg No: 240968040, Name: Devarjya
Enter elements of 5x5 matrix A:
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Resultant Matrix B:

0	1	1	1	1
5	0	1	1	1
5	5	0	1	1
5	5	5	0	1
5	5	5	5	0

Execution Time = 0.000026 seconds

```
indrath@indr4th ~-> export OMP_NUM_THREADS=2
indrath@indr4th ~-> ./q9
```

Reg No: 240968040, Name: Devarjya
Enter elements of 5x5 matrix A:

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Resultant Matrix B:

0	1	1	1	1
5	0	1	1	1
5	5	0	1	1
5	5	5	0	1
5	5	5	5	0

Execution Time = 0.000141 seconds

```
indrath@indr4th ~-> export OMP_NUM_THREADS=4
indrath@indr4th ~-> ./q9
```

Reg No: 240968040, Name: Devarjya
Enter elements of 5x5 matrix A:

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Resultant Matrix B:

0	1	1	1	1
5	0	1	1	1
5	5	0	1	1
5	5	5	0	1
5	5	5	5	0

Execution Time = 0.000274 seconds

```
indrath@indr4th ~-> export OMP_NUM_THREADS=8
indrath@indr4th ~-> ./q9
```

Reg No: 240968040, Name: Devarjya
Enter elements of 5x5 matrix A:

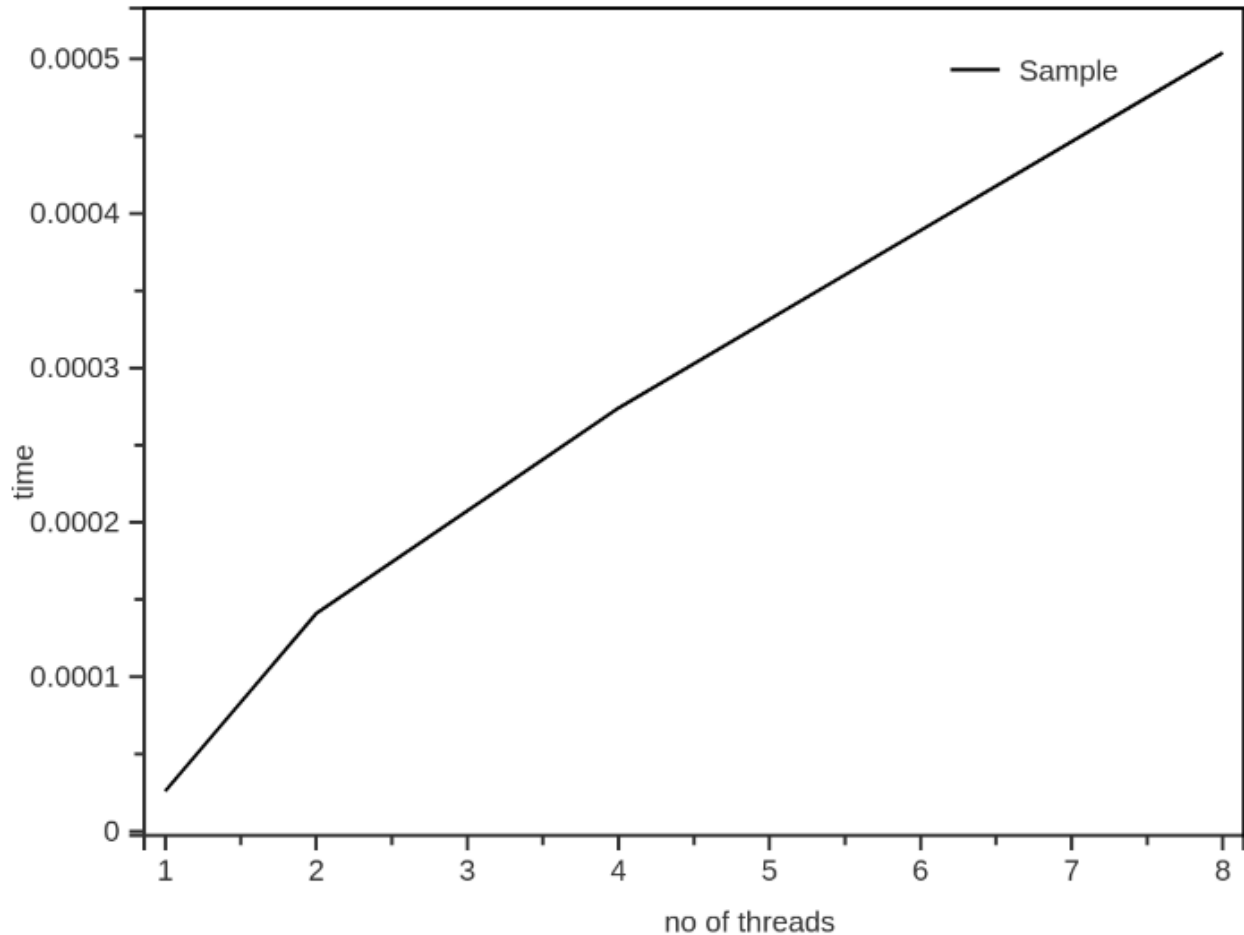
```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Resultant Matrix B:

0	1	1	1	1
5	0	1	1	1
5	5	0	1	1
5	5	5	0	1
5	5	5	5	0

Execution Time = 0.000504 seconds

Plot



10) Write a parallel program using OpenMP that reads a matrix of size $M \times N$ and produce an output matrix B of same size such that it replaces all the non-border elements of A with its equivalent 1's complement and remaining elements same as matrix A. Also produce a matrix D as shown below

Code:

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
void printComplement(int n, int *decimal)
```

```
{
```

```
    int bits[32], comp[32];
```

```
    int count = 0;
```

```
    while (n > 0)
```

```
    {
```

```
        bits[count++] = n % 2;
```

```
        n /= 2;
```

```
    }
```

```
    if (count == 0)
```

```
    {
```

```
        printf("1");
```

```

    *decimal = 1;
    return;
}

*decimal = 0;

for (int i = count - 1; i >= 0; i--)
{
    comp[i] = bits[i] ? 0 : 1;
    printf("%d", comp[i]);
}

int power = 1;
for (int i = 0; i < count; i++)
{
    *decimal += comp[i] * power;
    power *= 2;
}
}

int main()
{
    int M, N;

    printf("Registration No: 240968040, Name: Devariya Purkayastha\n");

    printf("Enter rows and columns: ");
    scanf("%d %d", &M, &N);

    int A[M][N], D[M][N];

    printf("Enter matrix:\n");
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &A[i][j]);

    double start = omp_get_wtime();

    printf("\nMatrix B:\n");

    #pragma omp parallel for collapse(2)
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (i == 0 || i == M - 1 || j == 0 || j == N - 1)
            {
                D[i][j] = A[i][j];

                #pragma omp critical

```

```

        printf("%d ", A[i][j]);
    }
    else
    {
        int dec;

        #pragma omp critical
        {
            printComplement(A[i][j], &dec);
            printf(" ");
        }

        D[i][j] = dec;
    }

    if (j == N - 1)
    {
        #pragma omp critical
        printf("\n");
    }
}

double end = omp_get_wtime();

printf("\nMatrix D:\n");
for (int i = 0; i < M; i++)
{
    for (int j = 0; j < N; j++)
        printf("%d ", D[i][j]);
    printf("\n");
}

printf("\nExecution Time = %f seconds\n", end - start);

return 0;
}

```

Output:

```
indrath@1ndr4th ~> export OMP_NUM_THREADS=4
indrath@1ndr4th ~> ./q10
Registration No: 240968040, Name: Devarjya Purkayastha
Enter rows and columns: 4 4
Enter matrix:
1 2 3 4
6 5 8 3
2 4 10 1
9 1 2 5

Matrix B:
9 6 010 0111 3
1 2 5
2 011 1 0101 1
2 3 4

Matrix D:
1 2 3 4
6 2 7 3
2 3 5 1
9 1 2 5

Execution Time = 0.000586 seconds
```

Q11) Write a parallel program in OpenMP to reverse the digits of the following integer array of size 9. Initialize the input array to the following values:

a. Input array: 18, 523, 301, 1234, 2, 14, 108, 150, 1928

b. Output array: 81, 325, 103, 4321, 2, 41, 801, 51, 8291

Code:

```
#include <stdio.h>
#include <omp.h>

int main()
{
    printf("Reg No: 240968040, Name: Devarjya\n");

    int arr[] = {18, 523, 301, 1234, 2, 14, 108, 150, 1928};
    int output[9];
    int size = 9;

    printf("Input: ");
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);

    double start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < size; i++)
    {
        int n = arr[i];
        int rev = 0;
```

```

    while (n > 0)
    {
        rev = rev * 10 + (n % 10);
        n /= 10;
    }

    output[i] = rev;
}

double end = omp_get_wtime();

printf("\nOutput: ");
for (int i = 0; i < size; i++)
    printf("%d ", output[i]);

printf("\nExecution Time = %f seconds\n", end - start);

return 0;
}

```

Output:

```

indrath@indr4th ~> export OMP_NUM_THREADS=4
indrath@indr4th ~> ./q11
Reg No: 240968040, Name: Devarjya
Input: 18 523 301 1234 2 14 108 150 1928
Output: 81 325 103 4321 2 41 801 51 8291
Execution Time = 0.000149 seconds

```